

Lumo

Make hard product decisions clearer, and keep a record of how they turned out.

Terrance Range · Solo build: product, design, engineering · www.trylumo.co

01

The short version

Lumo is for product managers who have to make a hard call and then explain it to everyone it touches.

You write down your first instinct in plain language. Lumo reads the situation back, surfaces options (including ones you may not have considered), and helps you commit with a written trade-off and a confidence rating. It then drafts a message for each audience that needs to hear the decision. The whole record is saved so the tool can later show you where your instincts have held up and where they have not.

I built it end to end as a work sample. It is a prototype, but a real one: live model calls, production deployment, English and Spanish.

02

The problem

AI gave you more options. It did not give you more judgment.

Product managers still make hard calls the same way they always have — quickly, under pressure, and mostly alone. Then they spend days translating that call for leadership, engineering, sales, and customers. Generative AI made the drafting faster. It did not make the decisions better, and it never remembered how last quarter's calls actually turned out.

Most tools start from zero every time. They help with the decision in front of you and then forget it. They rarely capture your first instinct before the analysis begins, and they almost never close the loop between what you believed and what happened.

That gap is the problem Lumo is built around.

Who it is for

A working product manager who regularly makes calls with real trade-offs: what to sequence, which date to promise, what to cut. Short on time, answerable to leadership, and stuck rewriting the same decision for different audiences.

Principles I set before writing any code

Capture the instinct first.

Ask for the raw take before any analysis, so there is something real to compare against later.

You commit, not the model.

The tool can structure, pressure-test, and draft. It does not choose.

Never fake it.

Real model output only. Sample content is clearly labeled.

Zero friction to start.

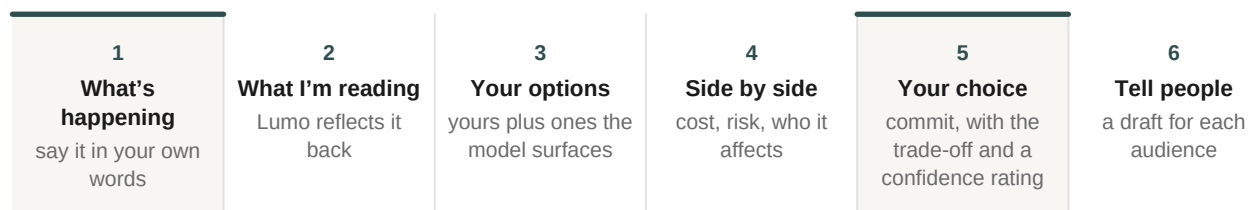
No account, no install, no approval.

Interrupt the next mistake.

Surface a pattern at the moment someone is about to repeat it, not in a monthly report.

The six-step flow

The product is one continuous path. The model helps in the middle. The first and fifth steps stay human.



Over time the system can say something like: you have been this sure about a date ten times. Seven of them slipped.

The decision that mattered most

When I first built Lumo, it recommended a decision. That seemed like the obvious useful feature. Within a couple of weeks I realized I hated it.

The moment the model told me what it would do, I stopped thinking and started reacting. The tool had quietly shifted from helping me reason to giving me an answer I could accept or reject. That was the opposite of the problem I was trying to solve.

So I removed the recommendation completely.

Claude can structure the situation, surface options I might have missed, and draft the follow-ups. It cannot make the call. Having to choose is the point of the product.

Two other constraints followed from the same logic. After you select an option you have to rate your confidence. A decision made at sixty percent should be treated differently from one made at ninety, and memory tends to erase that distinction. And the tool will not generate audience messages until you write a short rationale. That was the exact step I had skipped in real life, and I did not want the product to let me skip it again.

How I tested whether it actually helped

I did not want another tool that felt helpful in the moment and left no evidence it improved anything. Early tests focused on three questions:

- Did people change their initial instinct after seeing the structured options, or did they just confirm what they already believed?
- Did the confidence rating create any useful friction, or did users treat it as a formality?
- Where did the model produce options that sounded reasonable but were actually shallow or misframed?

The pattern that showed up most often: the model is good at expanding the set of options and at drafting clear messages. It is less reliable at surfacing the non-obvious downside or the political constraint that a human already feels but has not yet named. When the tool was most useful, it was usually because it forced the person to write the trade-off down, not because it had a better idea than they did.

That observation reinforced the original design choice. The value is not in the model's recommendation. The value is in the structure and the record.

What still does not work well

Known limitations

The model sometimes offers options that are plausible on the surface and weak on inspection. It can miss organizational reality — the stakeholder who will block the decision, the commitment that was already made quietly, the constraint that only exists in people's heads. Confidence ratings are easy to game or rush through. And because the current version stores everything in the browser, the longer-term pattern detection is still mostly illustrative rather than proven on real longitudinal data.

These are not edge cases I plan to ignore. They are the next set of problems the product has to confront if it is going to be trusted with real decisions.

How it works (briefly)

Nothing exotic. The effort went into making the model's output predictable and honest.

- Claude is called with forced tool use so the response has to match a schema.
- The server validates every draft before it reaches the screen. Bad output is rejected, not displayed.
- There is no account and no server-side storage of decisions. Everything stays in the browser.
- Rate limiting keeps cost under control on a public tool.
- The interface is available in English and Spanish.

When something fails, the product says so directly instead of showing a polished empty state.

What I would build next

- **Outcome check-ins** — prompt the user later to log how the decision actually landed.
- **Pattern detection on real history** — replace sample data with the user's own record.
- **Better handling of weak model options** — make it easier to flag and dismiss shallow suggestions.
- **Light team context** — so the tool can account for known constraints without requiring a full knowledge base.

Lumo is my attempt at the part most AI tools still skip: better judgment, and a memory of how it turned out.